

CSS NOTES

Table of Contents

- 1. CSS Introduction & Selectors
- 2. Advanced Selectors & Box Model
- 3. Colors, Typography & CSS Variables
- 4. CSS Backgrounds & Gradients
- 5. CSS Layouts – Display & Flexbox
- 6. CSS Grid & Units
- 7. Transitions, Transforms & Animations
- 8. Animations & Media Queries
- 9. Quick Revision Tables
- 10. Practice Tasks

1. CSS Introduction & Selectors

CSS: Cascading Style Sheets is a stylesheet language used to control the presentation, layout, colors, typography, spacing, and visual behavior of HTML elements.

Purpose: HTML provides structure, while CSS controls how that structure looks and behaves on different screen sizes.

1.1 History and CSS Versions

- CSS was introduced to separate webpage structure from presentation.
- CSS1 provided basic styling such as fonts, colors, margins, and alignment.
- CSS2 added positioning, media types, and more advanced layout features.
- CSS2.1 clarified and stabilized many CSS2 features.
- CSS3 introduced modular specifications such as Flexbox, Grid, transitions, transforms, animations, and media queries.
- Modern CSS is developed as separate modules rather than one single numbered version.

```
HTML → Structure  
CSS → Presentation  
JavaScript → Behavior
```

1.2 Ways to Apply CSS

Inline CSS

Definition: CSS written directly inside an HTML element using the style attribute.

Syntax

```
<p style="color: blue;">Hello</p>
```

Example

```
<p style="color: blue; font-size: 20px;">Hello</p>
```

Internal CSS

Definition: CSS written inside a <style> element in the HTML document.

Syntax

```
<style>  
  selector {  
    property: value;  
  }  
</style>
```

Example

```
<style>  
  h1 {  
    color: darkblue;  
  }  
</style>
```

External CSS

Definition: CSS written in a separate .css file and connected to HTML using <link>.

Syntax

```
<link rel="stylesheet" href="style.css">
```

Example

```
/* style.css */
h1 {
  color: darkblue;
}
```

1.3 Specificity and Cascade

Cascade: When multiple CSS rules target the same element, the browser decides which declaration applies using importance, origin, specificity, and source order.

Specificity: Specificity is the weight assigned to a selector to determine which matching rule has higher priority.

- Inline styles generally have higher specificity than normal author stylesheet rules.
- ID selectors have higher specificity than class, attribute, and pseudo-class selectors.
- Class, attribute, and pseudo-class selectors have higher specificity than element selectors.
- If specificity is equal, the rule written later normally wins.
- !important can override normal declarations, but should be used sparingly.

```
p {
  color: blue;
}

.note {
  color: green;
}

#message {
  color: red;
}

/* For <p id="message" class="note">Text</p>,
   the ID rule has greater specificity. */
```

1.4 Simple Selectors

Element / Tag Selector

Definition: Selects all elements of a particular HTML tag.

Syntax

```
h1 { property: value; }
```

Example

```
h1 {
  color: navy;
}
```

Class Selector

Definition: Selects all elements that contain a specified class attribute. It begins with a dot.

Syntax

```
.class-name { property: value; }
```

Example

```
.card {  
  padding: 20px;  
}
```

ID Selector

Definition: Selects the element with a specified id. It begins with #.

Syntax

```
#id-name { property: value; }
```

Example

```
#header {  
  background-color: black;  
}
```

1.5 Combinator Selectors

Descendant Combinator

Definition: Selects elements that are anywhere inside another element.

Syntax

```
ancestor descendant { property: value; }
```

Example

```
nav a {  
  text-decoration: none;  
}
```

Child Combinator

Definition: Selects only direct children of an element.

Syntax

```
parent > child { property: value; }
```

Example

```
ul > li {  
  margin-bottom: 8px;  
}
```

Adjacent Sibling Combinator

Definition: Selects the first element immediately following another element.

Syntax

```
element + element { property: value; }
```

Example

```
h2 + p {  
  margin-top: 0;  
}
```

General Sibling Combinator

Definition: Selects all matching siblings that appear after the first element.

Syntax

```
element ~ element { property: value; }
```

Example

```
h2 ~ p {  
  color: gray;  
}
```

2. Advanced Selectors & Box Model

2.1 Attribute Selectors

Basic Attribute Selector

Definition: Selects elements that contain a specified attribute.

Syntax

```
[attribute] { property: value; }
```

Example

```
input[required] {  
  border: 2px solid black;  
}
```

Attribute Equals

Definition: Selects elements whose attribute exactly matches a value.

Syntax

```
[attribute="value"] { property: value; }
```

Example

```
input[type="email"] {  
  background-color: #f5f5f5;  
}
```

Attribute Starts With

Definition: Selects elements whose attribute value begins with the given text.

Syntax

```
[attribute^="value"] { property: value; }
```

Example

```
a[href^="https"] {  
  color: green;  
}
```

Attribute Contains

Definition: Selects elements whose attribute value contains the given text.

Syntax

```
[attribute*="value"] { property: value; }
```

Example

```
a[href*="youtube"] {  
  font-weight: bold;  
}
```

Attribute Ends With

Definition: Selects elements whose attribute value ends with the given text.

Syntax

```
[attribute$="value"] { property: value; }
```

Example

```
a[href$=".pdf"] {  
  color: darkred;  
}
```

2.2 Pseudo-class Selectors

Pseudo-class: A pseudo-class selects an element based on a special state, position, or user interaction.

:hover

Definition: Applies styles when the pointer is over an element.

Syntax

```
selector:hover { property: value; }
```

Example

```
button:hover {  
  background-color: black;  
  color: white;  
}
```

:focus

Definition: Applies styles when an element has keyboard or input focus.

Syntax

```
selector:focus { property: value; }
```

Example

```
input:focus {  
  outline: 2px solid blue;  
}
```

:first-child

Definition: Selects an element when it is the first child of its parent.

Syntax

```
selector:first-child { property: value; }
```

Example

```
li:first-child {  
  font-weight: bold;  
}
```

:last-child

Definition: Selects an element when it is the last child of its parent.

Syntax

```
selector:last-child { property: value; }
```

Example

```
li:last-child {  
  margin-bottom: 0;  
}
```

:nth-child()

Definition: Selects an element based on its position among its siblings.

Syntax

```
selector:nth-child(n) { property: value; }
```

Example

```
li:nth-child(2) {  
  color: red;  
}
```

:not()

Definition: Selects elements that do not match the selector inside :not().

Syntax

```
selector:not(other-selector) { property: value; }
```

Example

```
button:not(.primary) {  
  border: 1px solid gray;  
}
```

2.3 Pseudo-elements

Pseudo-element: A pseudo-element styles a specific part of an element or creates generated content without adding another HTML element.

::before

Definition: Creates generated content before the element's actual content.

Syntax

```
selector::before { content: ""; }
```

Example

```
.required::before {  
  content: "*";  
  color: red;  
}
```

::after

Definition: Creates generated content after the element's actual content.

Syntax

```
selector::after { content: ""; }
```

Example

```
.external::after {  
  content: " ^";  
}
```

::first-line

Definition: Styles the first formatted line of a block-level element.

Syntax

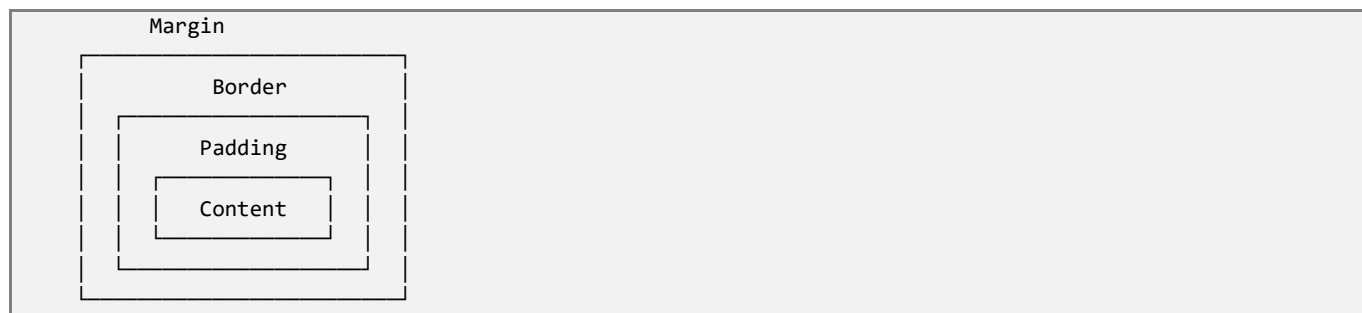
```
selector::first-line { property: value; }
```

Example

```
p::first-line {  
  font-weight: bold;  
}
```

2.4 CSS Box Model

Box Model: Every HTML element is treated as a rectangular box consisting of content, padding, border, and margin.



Content

Definition: The actual text, image, or child content area.

Syntax

```
width: value; height: value;
```

Example

```
.box {  
  width: 300px;  
  height: 150px;  
}
```

Padding

Definition: Space between the content and the border.

Syntax

```
padding: value;
```

Example

```
.box {  
  padding: 20px;  
}
```

Border

Definition: A visible or invisible line around the padding and content.

Syntax

```
border: width style color;
```

Example

```
.box {  
  border: 2px solid black;  
}
```

Margin

Definition: Space outside the border that separates the element from surrounding elements.

Syntax

```
margin: value;
```

Example

```
.box {  
  margin: 20px;  
}
```

box-sizing

Definition: Controls how width and height are calculated.

Syntax

```
box-sizing: border-box;
```

Example

```
* {  
  box-sizing: border-box;  
}
```

Overflow

Definition: Controls what happens when content is larger than the element's box.

Syntax

```
overflow: visible | hidden | scroll | auto;
```

Example

```
.card {  
  width: 250px;  
  height: 120px;  
  overflow: auto;  
}
```

2.5 Shorthand Properties

Shorthand: A shorthand property lets multiple related CSS properties be written in one declaration.

CSS Box Model – Shorthand Properties

CSS Box Model

Space outside the border (transparent)

Border around the padding and content

Space between border and content

Actual content (text, image, etc.)

Shorthand Syntax (General Rule)

```
property: top right bottom left;
```

Values:

- 1 value** → applies to all four sides (top, right, bottom, left)
- 2 values** → first value: top & bottom, second value: right & left
- 3 values** → first: top, second: right & left, third: bottom
- 4 values** → top, right, bottom, left (clockwise)

Margin Shorthand

The **margin** property sets the outer space of an element.

Syntax:
`margin: top right bottom left;`

Examples:

All sides same	<code>margin: 20px;</code>
Top & bottom Right & left	<code>margin: 20px 10px;</code>
Top Right & left Bottom	<code>margin: 20px 10px 30px;</code>
Top Right Bottom Left	<code>margin: 10px 20px 30px 40px;</code>
Auto margin (center)	<code>margin: 0 auto;</code>

Padding Shorthand

The **padding** property sets the inner space between border and content.

Syntax:
`padding: top right bottom left;`

Examples:

All sides same	<code>padding: 15px;</code>
Top & bottom Right & left	<code>padding: 15px 10px;</code>
Top Right & left Bottom	<code>padding: 15px 10px 25px;</code>
Top Right Bottom Left	<code>padding: 5px 10px 15px 20px;</code>

Border Shorthand

The **border** property sets the width, style and color of the border.

Syntax:
`border: width style color;`

Examples:

All sides same	<code>border: 2px solid #333;</code>
Different styles (top right bottom left)	<code>border: 2px solid #333 1px dashed #666 2px dotted #999 1px double #000;</code>
Only specific side	<code>border-top: 2px solid #333;</code>
No border	<code>border: none;</code>

Quick Reference – Value Pattern

Number of values	Pattern	Example (margin)	Applies to
1	value	<code>margin: 20px;</code>	top, right, bottom, left
2	vertical horizontal	<code>margin: 20px 10px;</code>	top & bottom right & left
3	top horizontal bottom	<code>margin: 20px 10px 30px;</code>	top right & left bottom
4	top right bottom left	<code>margin: 10px 20px 30px 40px;</code>	top right bottom left (clockwise)

Note:

- The same shorthand pattern works for margin, padding and border.
- For border, you must provide width, style and color.
- Order is always: top → right → bottom → left (clockwise).
- You can also use individual properties like margin-top, padding-left, border-right when needed.

```
/* Margin: top right bottom left */
margin: 10px 20px 30px 40px;

/* Padding */
padding: 10px 20px;

/* Border */
border: 2px solid black;

/* Background */
background: white url("image.jpg") center / cover no-repeat;
```

3. Colors, Typography & CSS Variables

3.1 CSS Color Formats

Named Colors

Definition: Uses predefined color names.

Syntax

```
color: color-name;
```

Example

```
color: tomato;
```

HEX

Definition: Represents red, green, and blue components using hexadecimal notation.

Syntax

```
color: #RRGGBB;
```

Example

```
color: #2563eb;
```

RGB

Definition: Defines a color using red, green, and blue numeric values.

Syntax

```
color: rgb(R, G, B);
```

Example

```
color: rgb(37, 99, 235);
```

RGBA

Definition: RGB with an alpha value for transparency.

Syntax

```
color: rgba(R, G, B, A);
```

Example

```
background-color: rgba(37, 99, 235, 0.2);
```

HSL

Definition: Defines a color using hue, saturation, and lightness.

Syntax

```
color: hsl(H, S, L);
```

Example

```
color: hsl(221, 83%, 53%);
```

3.2 Typography

font-family

Definition: Specifies the typeface used for text.

Syntax

```
font-family: family-name, fallback;
```

Example

```
font-family: "Arial", sans-serif;
```

font-size

Definition: Sets the size of text.

Syntax

```
font-size: value;
```

Example

```
font-size: 18px;
```

font-weight

Definition: Controls the thickness of text.

Syntax

```
font-weight: value;
```

Example

```
font-weight: 700;
```

font-style

Definition: Controls whether text is normal, italic, or oblique.

Syntax

```
font-style: value;
```

Example

```
font-style: italic;
```

line-height

Definition: Controls the vertical space between lines of text.

Syntax

```
line-height: value;
```

Example

```
line-height: 1.6;
```

letter-spacing

Definition: Controls the horizontal spacing between characters.

Syntax

```
letter-spacing: value;
```

Example

```
letter-spacing: 1px;
```

text-align

Definition: Controls horizontal alignment of inline content inside a block.

Syntax

```
text-align: value;
```

Example

```
text-align: center;
```

text-decoration

Definition: Adds or removes decorations such as underline or line-through.

Syntax

```
text-decoration: value;
```

Example

```
text-decoration: underline;
```

text-transform

Definition: Changes the capitalization of text.

Syntax

```
text-transform: value;
```

Example

```
text-transform: uppercase;
```

3.3 Google Fonts Integration

Definition: Google Fonts provides web fonts that can be included in a webpage using a stylesheet link or CSS import.

```
<head>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600&display=swap"
    rel="stylesheet">
</head>
body {
  font-family: "Poppins", sans-serif;
}
```

3.4 CSS Custom Properties / Variables

Definition: CSS custom properties are reusable values defined with a name beginning with -- and accessed with var().

```
:root {
  --primary: #2563eb;
  --spacing: 16px;
}

.button {
  background-color: var(--primary);
  padding: var(--spacing);
}
```

Fallback value: A fallback can be supplied if the custom property is unavailable.

```
color: var(--primary, black);
```

4. CSS Backgrounds & Gradients

background-color

Definition: Sets the background color.

Syntax

```
background-color: value;
```

Example

```
body {  
  background-color: #f8fafc;  
}
```

background-image

Definition: Places an image or gradient behind the content.

Syntax

```
background-image: url(...);
```

Example

```
hero {  
  background-image: url("hero.jpg");  
}
```

background-position

Definition: Controls where the background image is positioned.

Syntax

```
background-position: x y;
```

Example

```
.hero {  
  background-position: center;  
}
```

CSS background-position – Different Values and Output

Definition: The background-position property sets the starting position of a background image within an element.

Syntax: background-position: x y;

Values can be keywords (top, right, etc.), percentages, or length units (px, em, %).

1. top left (keyword)

background-position: top left;



Image positioned at the top left corner.

2. top center (keyword)

background-position: top center;



Image positioned at the top center.

3. top right (keyword)

background-position: top right;



Image positioned at the top right corner.

4. center (keyword)

background-position: center;



Image positioned at the center.

5. bottom left (keyword)

background-position: bottom left;



Image positioned at the bottom left corner.

6. bottom center (keyword)

background-position: bottom center;



Image positioned at the bottom center.

7. bottom right (keyword)

background-position: bottom right;



Image positioned at the bottom right corner.

8. 50% 50% (percentage)

background-position: 50% 50%;



Image positioned at 50% from left and 50% from top (center).

9. 20% 30% (percentage)

background-position: 20% 30%;



Image positioned 20% from left and 30% from top.

10. 100px 50px (length)

background-position: 100px 50px;



Image positioned 100px from left and 50px from top.

11. right 20px bottom 30px (mixed)

background-position: right 20px bottom 30px;



Image positioned 20px from right and 30px from bottom.

12. 0 0 (length)

background-position: 0 0;



Image positioned at 0px from left and 0px from top (top left corner).

Common Values

top, bottom, left, right	Keyword values
center	Centers the image
% values (0% – 100%)	Relative to the element size
px, em, rem, etc.	Fixed distance from top and left

Quick Examples

```
background-position: top left;
background-position: center;
background-position: 50% 50%;
background-position: 100px 50px;
background-position: right 20px bottom 30px;
background-position: 0 0;
```

Key Points

- The first value sets the horizontal position (x-axis).
- The second value sets the vertical position (y-axis).
- You can use keywords, percentages, or length units.
- If you use only one value, the other value defaults to center.
- The image is positioned inside the element's background area (not the content area).

background-size

Definition: Controls the size of the background image.

Syntax

```
background-size: value;
```

Example

```
.hero {
  background-size: cover;
}
```

background-repeat

Definition: Controls whether a background image repeats.

Syntax

```
background-repeat: value;
```

Example

```
.hero {  
  background-repeat: no-repeat;  
}
```

Linear Gradient

Definition: Creates a smooth color transition along a line.

Syntax

```
background: linear-gradient(direction, color1, color2);
```

Example

```
.banner {  
  background: linear-gradient(90deg, #2563eb, #7c3aed);  
}
```

Radial Gradient

Definition: Creates a color transition radiating from a center point.

Syntax

```
background: radial-gradient(shape, color1, color2);
```

Example

```
.circle {  
  background: radial-gradient(circle, #fef3c7, #f59e0b);  
}
```

Multiple Backgrounds

Definition: Allows more than one background image or gradient on the same element.

Syntax

```
background: image1, image2, color;
```

Example

```
.card {  
  background: url("pattern.png") center / cover, white;  
}
```

4.1 CSS Patterns Using Gradients

Definition: Repeated gradients can create simple patterns without separate image files.

```
.pattern {  
  background:  
    repeating-linear-gradient(  
      45deg,  
      #e5e7eb 0 10px,  
      #ffffff 10px 20px  
    );  
}
```

5. CSS Layouts – Display & Flexbox

5.1 Display Property

block

Definition: Starts on a new line and normally takes the available width.

Syntax

```
display: block;
```

Example

```
.box {  
  display: block;  
}
```

inline

Definition: Flows within text and normally does not accept width and height in the same way as block-level boxes.

Syntax

```
display: inline;
```

Example

```
span {  
  display: inline;  
}
```

inline-block

Definition: Flows inline while allowing box dimensions such as width and height.

Syntax

```
display: inline-block;
```

Example

```
.item {  
  display: inline-block;  
  width: 150px;  
}
```

none

Definition: Removes the element from the layout.

Syntax

```
display: none;
```

Example

```
.menu {  
  display: none;  
}
```

flex

Definition: Makes an element a flex container and lays out its direct children along a flex axis.

Syntax

```
display: flex;
```

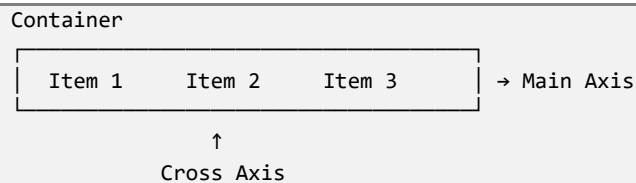
Example

```
.container {  
  display: flex;  
}
```

5.2 Flexbox

Flex Container: The parent element with display: flex.

Flex Item: A direct child of a flex container.



flex-direction

Definition: Defines the main axis and direction of flex items.

Syntax

```
flex-direction: row | row-reverse | column | column-reverse;
```

Example

```
.container {  
  display: flex;  
  flex-direction: row;  
}
```

flex-wrap

Definition: Controls whether flex items stay on one line or wrap onto multiple lines.

Syntax

```
flex-wrap: nowrap | wrap | wrap-reverse;
```

Example

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
}
```

justify-content

Definition: Aligns flex items along the main axis.

Syntax

```
justify-content: value;
```

Example

```
.container {  
  justify-content: space-between;  
}
```

align-items

Definition: Aligns flex items along the cross axis of the current flex line.

Syntax

```
align-items: value;
```

Example

```
.container {  
  align-items: center;  
}
```

align-content

Definition: Controls spacing between multiple flex lines when wrapping occurs.

Syntax

```
align-content: value;
```

Example

```
.container {  
  flex-wrap: wrap;  
  align-content: space-between;  
  height: 400px;  
}
```

gap

Definition: Sets the spacing between flex or grid items.

Syntax

```
gap: row-gap column-gap;
```

Example

```
.container {  
  display: flex;  
  gap: 20px;  
}
```

flex-grow

Definition: Controls how much an item can grow relative to other flex items.

Syntax

```
flex-grow: number;
```

Example

```
.item {  
  flex-grow: 1;  
}
```

flex-shrink

Definition: Controls how much an item can shrink when there is insufficient space.

Syntax

```
flex-shrink: number;
```

Example

```
.item {  
  flex-shrink: 1;  
}
```

flex-basis

Definition: Sets the initial main-size of a flex item before free space is distributed.

Syntax

```
flex-basis: value;
```

Example

```
.item {  
  flex-basis: 200px;  
}
```

6. CSS Grid & Units

6.1 CSS Grid

CSS Grid: A two-dimensional layout system that controls rows and columns.

grid-template-columns

Definition: Defines the number and size of grid columns.

Syntax

```
grid-template-columns: values;
```

Example

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
}
```

grid-template-rows

Definition: Defines the number and size of grid rows.

Syntax

```
grid-template-rows: values;
```

Example

```
.grid {  
  display: grid;  
  grid-template-rows: 100px 200px;  
}
```

grid-area

Definition: Places a grid item using a named area or row/column line values.

Syntax

```
grid-area: value;
```

Example

```
.header {  
  grid-area: header;  
}
```

fr unit

Definition: Represents a fraction of the available space in a grid container.

Syntax

```
grid-template-columns: 1fr 2fr;
```

Example

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr 2fr;  
}
```

auto-fit

Definition: Creates as many columns as fit in the available space, allowing tracks to collapse when necessary.

Syntax

```
repeat(auto-fit, minmax(...));
```

Example

```
.grid {  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
}
```

auto-fill

Definition: Creates as many tracks as can fit, preserving available track slots even when fewer items are present.

Syntax

```
repeat(auto-fill, minmax(...));
```

Example

```
.grid {  
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));  
}
```

6.2 CSS Units

Unit	Meaning	Example
px	Absolute CSS pixel unit commonly used for precise sizes.	font-size: 16px;
em	Relative to the font size of the relevant element; for many properties it is	padding: 1em;

	based on the element's computed font size.	
rem	Relative to the root element's font size.	font-size: 1.5rem;
%	Relative to a relevant containing dimension or property context.	width: 50%;
vh	1% of the viewport height.	height: 100vh;
vw	1% of the viewport width.	width: 100vw;

6.3 CSS Functions

calc()

Definition: Performs calculations to determine a CSS value.

Syntax

```
property: calc(expression);
```

Example

```
.main {
  width: calc(100% - 40px);
}
```

min()

Definition: Uses the smallest value from a list of values.

Syntax

```
property: min(value1, value2);
```

Example

```
.box {
  width: min(90%, 900px);
}
```

max()

Definition: Uses the largest value from a list of values.

Syntax

```
property: max(value1, value2);
```

Example

```
.box {
  padding: max(16px, 2vw);
}
```

clamp()

Definition: Sets a minimum, preferred, and maximum value.

Syntax

```
property: clamp(min, preferred, max);
```

Example

```
h1 {  
  font-size: clamp(2rem, 5vw, 4rem);  
}
```

7. Transitions, Transforms & Animations

7.1 Transitions

Transition: A transition smoothly changes a CSS property from one value to another over a specified duration.

transition-property

Definition: Specifies which property should transition.

Syntax

```
transition-property: property;
```

Example

```
button {  
  transition-property: background-color;  
}
```

transition-duration

Definition: Specifies how long the transition takes.

Syntax

```
transition-duration: time;
```

Example

```
button {  
  transition-duration: 0.3s;  
}
```

transition-timing-function

Definition: Controls the speed curve of the transition.

Syntax

```
transition-timing-function: ease | linear | ease-in | ease-out | ease-in-out | cubic-bezier(...);
```

Example

```
button {  
  transition-timing-function: ease;  
}
```

transition

Definition: Shorthand for common transition properties.

Syntax

```
transition: property duration timing-function delay;
```

Example

```
button {  
  transition: transform 0.3s ease;  
}  
  
.button {  
  background-color: navy;  
  transition: background-color 0.3s ease,  
             transform 0.3s ease;  
}  
  
.button:hover {  
  background-color: royalblue;  
  transform: scale(1.05);  
}
```

7.2 Transform

Transform: The transform property visually changes an element's position, size, angle, or shape without changing the normal document flow.

translate

Definition: Moves an element horizontally and/or vertically.

Syntax

```
transform: translate(x, y);
```

Example

```
.box {  
  transform: translate(20px, 10px);  
}
```

translateX / translateY / translateZ

Definition: Moves an element along the X, Y, or Z axis.

Syntax

```
transform: translateX(value);
```

Example

```
.box {  
  transform: translateX(50px);  
}
```

scale

Definition: Changes the visual size of an element.

Syntax

```
transform: scale(x, y);
```

Example

```
.card:hover {  
  transform: scale(1.05);  
}
```

scaleX / scaleY / scaleZ

Definition: Scales an element along the X, Y, or Z axis.

Syntax

```
transform: scaleX(value);
```

Example

```
.box {  
  transform: scaleX(1.2);  
}
```

rotate

Definition: Rotates an element around its transform origin.

Syntax

```
transform: rotate(angle);
```

Example

```
.icon {  
  transform: rotate(15deg);  
}
```

skew

Definition: Slants an element along the X and/or Y axis.

Syntax

```
transform: skew(x-angle, y-angle);
```

Example

```
.banner {  
  transform: skew(-10deg);  
}
```

Combining Transforms

Definition: Multiple transform functions can be applied in one transform declaration.

Syntax

```
transform: translate(...) rotate(...) scale(...);
```

Example

```
.card:hover {  
  transform: translateY(-8px) rotate(1deg) scale(1.02);  
}
```

8. Animations & Media Queries

8.1 @keyframes

@keyframes: Defines the stages of a CSS animation by specifying what styles should apply during the animation.

```
@keyframes slideIn {  
  from {  
    transform: translateX(-100px);  
    opacity: 0;  
  }  
  
  to {  
    transform: translateX(0);  
    opacity: 1;  
  }  
}
```

8.2 Animation Properties

animation-name

Definition: Specifies the name of the @keyframes animation.

Syntax

```
animation-name: name;
```

Example

```
.box {  
  animation-name: slideIn;  
}
```

animation-duration

Definition: Specifies how long one animation cycle takes.

Syntax

```
animation-duration: time;
```

Example

```
.box {  
  animation-duration: 1s;  
}
```

animation-timing-function

Definition: Controls the speed curve of the animation.

Syntax

```
animation-timing-function: value;
```

Example

```
.box {  
  animation-timing-function: ease;  
}
```

animation-delay

Definition: Specifies how long to wait before starting the animation.

Syntax

```
animation-delay: time;
```

Example

```
.box {  
  animation-delay: 0.5s;  
}
```

animation-iteration-count

Definition: Controls how many times the animation runs.

Syntax

```
animation-iteration-count: number | infinite;
```

Example

```
.loader {  
  animation-iteration-count: infinite;  
}
```

animation-direction

Definition: Controls whether the animation runs forward, reverse, or alternates.

Syntax

```
animation-direction: value;
```

Example

```
.box {  
  animation-direction: alternate;  
}
```

animation-fill-mode

Definition: Controls styles applied before and/or after animation execution.

Syntax

```
animation-fill-mode: value;
```

Example

```
.box {  
  animation-fill-mode: forwards;  
}
```

animation

Definition: Shorthand for animation properties.

Syntax

```
animation: name duration timing-function delay iteration-count direction fill-mode;
```

Example

```
.box {  
  animation: slideIn 1s ease 0s 1 normal forwards;  
}
```

8.3 Practical Animation: Loader

```
.loader {  
  width: 40px;  
  height: 40px;
```

```
border: 5px solid #ddd;
border-top-color: #2563eb;
border-radius: 50%;
animation: spin 1s linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}
```

8.4 Practical Animation: Banner

```
.banner {
  animation: bannerIn 1s ease;
}

@keyframes bannerIn {
  from {
    opacity: 0;
    transform: translateY(-30px);
  }

  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

8.5 Media Queries

Media Query: A media query applies CSS only when specified conditions, such as viewport width, are true.

```
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
```

Breakpoint: A breakpoint is a viewport condition where the layout or styling changes to better fit the available screen size.

Mobile-first: Start with styles for smaller screens, then add styles for larger screens using min-width media queries.

```
/* Base: mobile */
.card {
  width: 100%;
}

/* Larger screens */
@media (min-width: 768px) {
  .card {
    width: 48%;
  }
}
```

Desktop-first: Start with desktop styles, then use max-width media queries to adapt the layout for smaller screens.

```
/* Base: desktop */
.container {
  display: flex;
}
```

```

/* Smaller screens */
@media (max-width: 768px) {
  .container {
    display: block;
  }
}

```

9. Quick Revision Tables

9.1 Selector Quick Reference

Selector Type	Example	Meaning
Element	p	All <p> elements
Class	.card	Elements with class="card"
ID	#header	Element with id="header"
Descendant	.card p	Any p inside .card
Child	.card > p	Direct p child of .card
Adjacent sibling	h2 + p	First p immediately after h2
General sibling	h2 ~ p	All p siblings after h2
Attribute	input[type="email"]	Inputs with type=email
Pseudo-class	button:hover	Button while hovered
Pseudo-element	p::first-line	First line of p

9.2 Flexbox: Parent vs Child

Applied To	Properties	Purpose
Flex Container / Parent	display, flex-direction, flex-wrap, justify-content, align-items, align-content, gap	Controls overall arrangement of direct children
Flex Item / Child	flex-grow, flex-shrink, flex-basis, order, align-self	Controls an individual item's behavior

9.3 Main Axis and Cross Axis

- flex-direction: row → main axis is horizontal; cross axis is vertical.
- flex-direction: column → main axis is vertical; cross axis is horizontal.
- justify-content → aligns items along the main axis.
- align-items → aligns items along the cross axis.
- align-content → controls multiple flex lines along the cross-axis direction.

10. Practice Tasks

- Create a product card using class, attribute, :hover, and ::before selectors.
- Create a navigation bar using Flexbox with space-between and gap.
- Create a responsive card layout using CSS Grid and repeat(auto-fit, minmax()).
- Build a pricing section using CSS variables for primary color, spacing, and border radius.
- Create a profile card using transform: scale() and transition.
- Create a notification badge using ::after.
- Create a loading spinner using @keyframes and border styling.
- Create a responsive webpage using a mobile-first media query.
- Create a hero banner using a background image, background-size: cover, and a gradient overlay.
- Create a dashboard layout using Grid with named grid areas.
